

lab-observability-api

How the C# code works — structure, syntax, and the journey a request takes through the files

Generated 2026-06-16 · covers everything under `src/lab-observability-api/`

The whole app in one sentence, for a 10-year-old: This program is a **translator robot**. A person asks it a question in plain English. The robot doesn't know the answer itself — it re-packages the question into a special format, hands it to a much smarter friend named **Claude** over the internet, waits for the answer, unpacks it, and hands it back in plain English. Every file in this project is either part of that translator robot's body, or a tool it uses to stay safe and keep notes about what it did.

Contents

- 1. The big picture
- 2. The folder map
- 3. C# syntax cheat sheet (using real code from this project)
- 4. File-by-file walkthrough
 - 4.1 Program.cs — the ignition switch
 - 4.2 The "shapes" — Models, Contracts, Options
 - 4.3 The seam — IChatModelProvider / IBatchChatModelProvider
 - 4.4 Controllers — the front door
 - 4.5 Services/AI — the orchestrators
 - 4.6 Services/Claude — the actual phone call
 - 4.7 Middleware, Extensions, Telemetry — the helpers
- 5. Three complete journeys, step by step
- 6. How pieces find each other (Dependency Injection)
- 7. Glossary — every C# word used in this project

1. The big picture

Three things can happen to a question sent to this app, and each one is a different-shaped conversation with Claude:

Ask & wait

"What's 2+2?" → wait → get the full answer at once
Sync chat

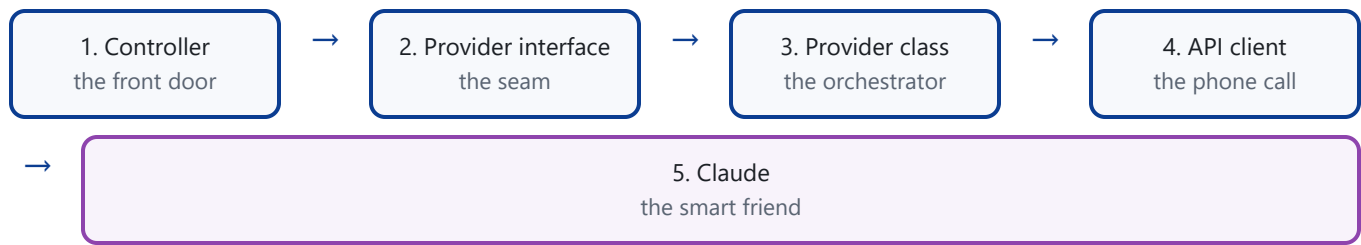
Ask & watch it type

Get the answer one word at a time, as it's written
Streaming chat

Drop off a stack

Submit 100 questions, come back hours later for all the answers, pay half price
Batch

All three paths funnel through the same five layers:



Layer 2 (the "seam") is the most important idea in the whole codebase. It's a **promise** — an interface — that says "whatever sits in layer 3 will be able to answer a chat request," without saying *how*. Today layer 3 is `ClaudeChatModelProvider`. If this company ever wants to talk to OpenAI or Amazon's Bedrock instead, only layer 3 and 4 change — the controller in layer 1 never has to know or care.

2. The folder map

Folder	What lives there, in plain words
<code>Controllers/</code>	The front doors. Each public web address (like <code>/api/ai/chat</code>) is a method in here.
<code>Services/AI/</code>	The promise (interfaces) and the orchestrator classes that fulfill the promise for Claude specifically.
<code>Services/Claude/</code>	The part that actually opens an internet connection to Claude and speaks its exact language.
<code>Models/AI/</code>	The "shapes" of data flowing in and out — what a chat request looks like, what a batch job looks like.
<code>Contracts/</code>	The shape of an error message, used everywhere something goes wrong.
<code>Options/</code>	The settings knobs — API key, which model, how long a prompt is allowed to be.
<code>Middleware/</code>	Code that runs on <i>every single request</i> before it reaches a controller — like a security guard at the front door.
<code>Extensions/</code>	Small helper methods bolted onto types ASP.NET already gives you.
<code>Telemetry/</code>	The clipboard — every counter and stopwatch the app uses to track its own behavior.
<code>Program.cs</code>	One file, sitting alone at the root. The ignition switch — wires every other piece together and starts the engine.

3. C# syntax cheat sheet (using real code from this project)

Every concept below is explained with an actual line pulled from this codebase — not a made-up textbook example.

namespace and using

Think of `namespace` as a street address for a piece of code, so two classes named the same thing (in different buildings) never collide. `using` is "I'm going to mention that street a lot, let me drop the full address and just use the building name."

```
namespace Lab.Observability.Api.Controllers;    // "this file lives in this neighborhood"

using Lab.Observability.Api.Services.AI;        // "I'm going to use things from that neighborhood"
```

class vs. interface vs. record

class = an actual robot you can build and use. **interface** = a job posting — a list of things any robot hired for this job must be able to do, with no robot inside it. **record** = a class built specifically to just *hold data*, like a labeled box, with free comparison ("are these two boxes the same?") built in.

```
public interface IChatModelProvider           // the job posting
{
    Task<ChatResponse> SendAsync(ChatRequest request, CancellationToken cancellationToken = default);
}

public sealed class ClaudeChatModelProvider : IChatModelProvider // the robot hired for the job
{ /* ... */ }

public record ChatChunk(string TextDelta, string? StopReason, ChatChunkUsage? Usage); // a labeled box
```

Properties: { get; set; }

A property is a labeled slot on a box that you can both read from and write to, written as a single line instead of two separate methods.

```
public class ChatRequest
{
    public string Prompt { get; set; } = string.Empty; // readable AND writable, starts as ""
}
```

Constructors and Dependency Injection

A constructor is what runs the moment a robot is built — usually to hand it the tools it'll need. In this project, nobody calls `new ClaudeChatModelProvider(...)` by hand anywhere — ASP.NET's container builds it automatically and hands it whatever the constructor asks for. See §6 for exactly how that works.

```
public sealed class ClaudeChatModelProvider : IChatModelProvider
{
    private readonly ClaudeApiClient _claudioApiClient; // a tool this robot keeps for itself

    public ClaudeChatModelProvider(ClaudeApiClient claudioApiClient, /* ...more tools... */)
    {
        _claudioApiClient = claudioApiClient; // "thanks, I'll keep this"
    }
}
```

`async` / `await` / `Task<T>`

Some jobs take time — like calling Claude over the internet. `async` marks a method as "this might have to wait around." `await` means "pause right here, let other work happen, and come back the instant this is ready." `Task<string>` means "I promise to eventually hand you a string — not yet, but eventually."

```
public async Task<ChatResponse> SendAsync(ChatRequest request, CancellationToken cancellationToken = default)
{
    // ...
    var responseText = await _claudioApiClient.SendChatAsync(payload, cancellationToken);
    return new ChatResponse { Provider = "anthropic", Response = responseText };
}
```

`IEnumerable<T>` and `yield return`

This is the syntax behind "ask & watch it type." Instead of promising ONE thing eventually (`Task<T>`), this promises a whole *stream* of things, arriving one at a time. `yield return` means "here's the next piece — pause me here, and pick back up exactly where I left off when you ask for the next one."

```
public async IEnumerable<ChatChunk> StreamAsync(ChatRequest request, CancellationToken ct)
{
    await foreach (var chunk in _claudioApiClient.StreamChatAsync(payload, ct))
    {
        // do something with each chunk as it arrives...
        yield return chunk; // hand this one piece to whoever is listening, then keep going
    }
}
```

Nullable types: `string?` and the `?.` / `??` operators

`string` promises "there will always be text here." `string?` admits "this might be empty — there might be nothing here at all," and the compiler forces you to check before you use it. `?.` means "do this, but only if the thing on the left actually exists." `??` means "use this value, but if it's missing, use this fallback instead."

```
public string? ProviderErrorCode { get; }    // might be null – "we don't always know the error code"

activity?.SetTag("llm.tokens.input", inputTokens.Value); // only set the tag if "activity" isn't null

cacheReadTokens ?? 0 // "use cacheReadTokens, or 0 if it's null"
```

String interpolation: `$"..."`

A normal sentence with blanks to fill in, like a fill-in-the-blank worksheet.

```
throw new ClaudeProviderException(message: $"Claude provider returned {(int)response.StatusCode}.");
```

Pattern matching: `switch` expressions

A lookup table written sideways: "if the value matches THIS, the answer is THAT" — for every case, ending in a catch-all.

```
context.Response.StatusCode = ex.ProviderStatusCode switch
{
    HttpStatusCode.TooManyRequests => StatusCodes.Status503ServiceUnavailable,
    HttpStatusCode.RequestTimeout   => StatusCodes.Status504GatewayTimeout,
    _ when ex.IsTransient           => StatusCodes.Status503ServiceUnavailable, // "_" = anything else, but
    _                               => StatusCodes.Status502BadGateway           // the true catch-all
};
```

Exceptions: `try` / `catch` / `throw`

"Attempt this. If something specific goes wrong, here's what to do about it instead of crashing." `throw` is raising your hand and saying "something is wrong, stop what you're doing and let someone further up handle it."

```
try
{
    var response = await _httpClient.PostAsJsonAsync("messages", anthropicRequest, cancellationToken);
}
catch (HttpRequestException ex)
{
    throw new ClaudeProviderException(message: "Claude provider request failed at the network level.", in
}
```

Attributes: `[ApiController]`, `[HttpPost]`

A sticky note attached to a class or method that tells ASP.NET "treat this specially" — without changing the code itself.

```
[ApiController] // "this class handles web requests, validate automatically"
[Route("api/[controller]")] // "my web address starts with api/ai" (controller name minus "Con
public class AiController : ControllerBase
{
    [HttpPost("chat")] // "this method answers POST /api/ai/chat"
    public async Task<ActionResult<ChatResponse>> Chat([FromBody] ChatRequest request, CancellationToken
}
```

Generics: `<T>`

A container shape that works for any type of thing you put inside it — like a labeled shoebox that works whether you put shoes, photos, or letters in it. You decide the contents when you use the box, not when you build it.

```
Task<ChatResponse> // "a promise of a ChatResponse, eventually"
IReadOnlyList<ChatRequest> // "a list of ChatRequests you're only allowed to read, not change"
Histogram<double> // "a measuring tool that records double-precision numbers"
```

Reading JSON without a fixed shape: `JsonDocument`

Anthropic's reply doesn't always look exactly the same shape every time (error formats vary, usage formats changed between API versions). Instead of forcing it into a rigid C# class, the code opens it like a tree and asks "do you have a branch called X? Follow it if so, ignore it if not" — safe even when the shape shifts underneath it.

```
using var document = JsonDocument.Parse(responseBody);
if (document.RootElement.TryGetProperty("content", out var contentElement) &&
    contentElement.ValueKind == JsonValueKind.Array)
{
    // walk the array, pull out the "text" pieces...
}
```


4. File-by-file walkthrough

4.1 `Program.cs` — the ignition switch

`src/lab-observability-api/Program.cs`

This is the very first thing that runs. Imagine setting up a food truck before opening: plug in the generator (logging), put up the menu board (Swagger), hire your staff and tell each of them their job (dependency injection — "you handle Claude calls," "you handle batch calls"), put a bouncer at the door (correlation ID middleware, exception handling), and then flip the "OPEN" sign (`app.Run()`).

It runs top to bottom, once, at startup — not per-request. Key moves, in order:

Step	Real code	Why
1	<code>builder.Host.UseSerilog(...)</code>	Replace the default logger with Serilog so every log line is structured (has named fields, not just a sentence)
2	<code>builder.Services.Configure<AnthropicOptions>(...)</code>	Bind the "Anthropic" section of <code>appsettings.json</code> into a real C# object
3	<code>builder.Services.AddOpenTelemetry() + UseAzureMonitor()</code>	Wire one single pipeline that exports traces/metrics/logs to Azure Application Insights
4	<code>AddHttpClient<ClaudeApiClient>(...).AddStandardResilienceHandler(...)</code>	Create the HTTP client used for sync/streaming chat, with timeouts and a circuit breaker (retry deliberately disabled — see §4.6)
5	<code>builder.Services.AddScoped<IChatModelProvider, ClaudeChatModelProvider>()</code>	"Whenever any code asks for the job-posting <code>IChatModelProvider</code> , hand it a <code>ClaudeChatModelProvider</code> " — this one line is the provider abstraction
6	Repeat 4–5 for <code>ClaudeBatchApiClient</code> / <code>IBatchChatModelProvider</code>	Same pattern, separate client, deliberately no resilience handler (retrying a batch submit could create a duplicate paid job)
7	<code>app.UseMiddleware<CorrelationIdMiddleware>()</code>	Every request gets tagged with a tracking number before anything else happens
8	<code>app.Use(async (context, next) => { try { await next(); } catch (...) {...} })</code>	The global safety net — if any controller throws an exception, this catches it here instead of letting raw error details leak to the caller
9	<code>app.MapGet("/health", ...) , /health/ready , etc.</code>	Simple endpoints written directly in <code>Program.cs</code> (not a controller) — "minimal API" style, fine for tiny endpoints with no real logic
10	<code>app.MapControllers()</code>	"Now go find every class marked <code>[ApiController]</code> and wire up their <code>[HttpGet]</code> / <code>[HttpPost]</code> methods as real web addresses"
11	<code>app.Run()</code>	Flip the open sign — start actually listening for requests

4.2 The "shapes" — Models, Contracts, Options

These files don't *do* anything — no logic, mostly just labeled boxes (properties) describing what data looks like as it moves between layers. C# calls this kind of class a "DTO" (data transfer object) when it's plain like this.

File	Shape of	Notable fields
<code>Models/AI/ChatRequest.cs</code>	What a caller sends in	<code>string Prompt</code>
<code>Models/AI/ChatResponse.cs</code>	What a sync chat call returns	<code>Provider</code> , <code>Model</code> , <code>Response</code>
<code>Models/AI/ChatChunk.cs</code> (a <code>record</code>)	One piece of a streaming reply	<code>TextDelta</code> (the new text), <code>StopReason?</code> (only set on the last chunk), <code>Usage?</code> (token counts, only on the last chunk)
<code>Models/AI/BatchJob.cs</code>	What you get back right after submitting a batch	<code>Id</code> , <code>Status</code> , <code>RequestCount</code> , <code>CreatedAt</code> , <code>ExpiresAt</code>
<code>Models/AI/BatchJobStatus.cs</code>	What polling a batch returns	<code>SucceededCount</code> , <code>ErroredCount</code> , <code>CanceledCount</code> , <code>ExpiredCount</code>
<code>Models/AI/BatchProcessingStatus.cs</code> (an <code>enum</code>)	The three states a batch can be in	<code>InProgress</code> , <code>Canceling</code> , <code>Ended</code>
<code>Models/AI/BatchResult.cs</code>	One answer inside a finished batch	<code>CustomId</code> , <code>IsSuccess</code> , <code>Response?</code> , <code>ErrorMessage?</code>
<code>Contracts/ApiError.cs</code> (a <code>record</code>)	Every error this app ever returns to a caller, in any situation	<code>Code</code> , <code>Message</code> , <code>CorrelationId?</code> — exactly 3 fields, on purpose, so no internal detail (like a stack trace) ever leaks out
<code>Options/AnthropicOptions.cs</code>	Everything configurable about how this app talks to Claude	<code>ApiKey</code> , <code>Model</code> (default <code>claude-sonnet-4-6</code>), <code>BaseUrl</code> , <code>MaxTokens</code> , <code>EnablePromptCaching</code> , <code>SystemPrompt</code> , <code>MaxBatchSize</code> (default 100), <code>MaxPromptLength</code> (default 32,000 characters)

An `enum` like `BatchProcessingStatus` is a fixed, named list of choices — like a multiple-choice question where only three answers are allowed and the compiler won't let you write a fourth one by accident.

4.3 The seam — `IChatModelProvider` / `IBatchChatModelProvider`

`Services/AI/IChatModelProvider.cs` | `Services/AI/IBatchChatModelProvider.cs`

These two files are pure job postings — no actual robot inside. They exist so that `AiController` and `AiBatchController` never have to know the word "Claude" or "Anthropic" at all. They only know "give me something that can `SendAsync`" or "give me something that can `SubmitBatchAsync`." Today the hired robot is Claude-shaped; tomorrow it could be OpenAI-shaped, and the controllers wouldn't need a single line changed.

```
public interface IChatModelProvider
{
    Task<ChatResponse> SendAsync(ChatRequest request, CancellationToken cancellationToken = default);

    // A *default* implementation — any provider gets this for free unless it overrides it.
    // A provider that can't truly stream just calls SendAsync once and hands back one chunk.
    async IAsyncEnumerable<ChatChunk> StreamAsync(ChatRequest request, CancellationToken ct)
    {
        var response = await SendAsync(request, ct);
        yield return new ChatChunk(response.Response, "end_turn", null);
    }
}
```

`IBatchChatModelProvider` is the same idea for the batch world — three jobs: `SubmitBatchAsync`, `GetBatchStatusAsync`, `GetBatchResultsAsync`, plus a `ProviderName` property. It's a *separate* interface rather than three more methods bolted onto `IChatModelProvider` — see ADR-010 — because batch isn't "chat, but slower," it's a fundamentally different shape of work (submit → wait → poll → retrieve, with its own ID and its own failure rules).

4.4 Controllers — the front door

Controllers/AiController.cs

A controller is the receptionist. It checks your ID (validates the request), walks your question back to the right department (`IChatModelProvider`), and brings the answer back to the front desk. It is deliberately "thin" — no real thinking happens here.

`POST /api/ai/chat` — `Chat(...)` method: check the prompt isn't empty, check it isn't longer than `MaxPromptLength`, call `_provider.SendAsync(...)`, return the result wrapped in `Ok(...)` (HTTP 200).

`POST /api/ai/chat/stream` — `StreamChat(...)` method: same two checks, then it does something a normal endpoint doesn't —

```
HttpContext.Features.Get<IHttpResponseBodyFeature>()?.DisableBuffering(); // tell the server "don't
Response.ContentType = "text/event-stream; charset=utf-8"; // "I'm sending a live

await foreach (var chunk in _provider.StreamAsync(request, HttpContext.RequestAborted))
{
    await Response.WriteAsync($"data: {JsonSerializer.Serialize(chunk, _sseJsonOptions)}\n\n", ...);
    await Response.Body.FlushAsync(...); // push it out the door right now, don't wait to bundle m
}
```

If something breaks mid-stream, the response code can't change anymore (the 200 already went out) — so instead it writes a special `event: error` line into the same stream, and the browser/caller is expected to watch for that.

Controllers/AiBatchController.cs

Three endpoints, all under `api/ai/batch`: `POST` (submit a list of `ChatRequest`), `GET /{id}` (poll status), `GET /{id}/results` (retrieve finished answers). `Submit` has four separate guard checks stacked in a row — empty list, any blank prompt, any prompt too long, too many requests at once — each returning a specific, named error code (`invalid_request`, `prompt_too_long`, `batch_size_exceeded`) before anything is sent to Claude.

Controllers/TestController.cs

The simplest file in the project — two endpoints, `GET /api/test/ping` (says "pong" back) and `GET /api/test/error` (deliberately throws an exception). Built on Day 4 to prove the very first Azure deployment actually worked and that errors show up in Application Insights. Superseded in spirit by the real `/health` endpoints added later, but still present and working.

4.5 Services/AI — the orchestrators

Services/AI/ClaudeChatModelProvider.cs

This is the robot hired for the `IChatModelProvider` job. Its job is the "business logic" of a chat turn: start a stopwatch, build the exact JSON shape Claude expects, ask the API client to actually send it, write down what happened, hand back the result. It never opens an internet connection itself — it delegates that to `ClaudeApiClient`.

`SendAsync` : validate the request isn't null/empty → start an OpenTelemetry "activity" (a labeled timer/trace span named `ai.chat.complete`) → build a small anonymous object shaped like `{ model, max_tokens, messages: [{ role: "user", content: prompt }] }` → await `_claudeApiClient.SendChatAsync(...)` → wrap the text that comes back into a `ChatResponse`.

`StreamAsync` : same idea, but `stream = true` in the payload, and instead of waiting for one full answer, it loops with `await foreach` over chunks coming from `_claudeApiClient.StreamChatAsync(...)`. The very first chunk gets a stopwatch reading recorded into `GatewayTelemetry.StreamTtftMs` — "time to first token," the metric this whole streaming feature exists to make small. A `try/finally` around the loop makes sure that even if the connection drops mid-stream, a log line still gets written saying so — so nothing fails silently.

Services/AI/ClaudeBatchChatModelProvider.cs

The shortest "real" class in the project — 17 lines. It's hired for the `IBatchChatModelProvider` job, but it does no actual work itself; every method is a single line that just forwards the call straight to `ClaudeBatchApiClient`. This exists purely so the seam (§4.3) stays satisfied — the controller depends on the interface, not on `ClaudeBatchApiClient` directly.

4.6 Services/Claude — the actual phone call

Services/Claude/ClaudeApiClient.cs (the biggest, busiest file in the project)

If `ClaudeChatModelProvider` is the person deciding what to say, `ClaudeApiClient` is the actual telephone — it's the only piece of code that ever opens a real network connection to `api.anthropic.com`.

SendChatAsync — the synchronous path:

1. Start a trace span named `claude.chat.api`, tag it with the provider/model name
2. Call `BuildAnthropicRequest(payload)` — adds the `system` prompt to the JSON, and if prompt caching is on, wraps it with `cache_control: { type: "ephemeral", ttl: "1h" }` so Claude doesn't re-process the same long instructions every time
3. `await _httpClient.PostAsJsonAsync("messages", anthropicRequest, ...)` — the actual HTTP call
4. If the response wasn't successful, dig the real error message and code out of the JSON body, decide whether the failure is *transient* (worth retrying later — 429, 408, 502, 503, 504) and throw a `ClaudeProviderException` carrying all of that
5. If it succeeded, pull the answer text and the token-usage numbers out of the JSON, record them as metrics (`CacheHits` / `CacheMisses` / `ProviderLatencyMs`), and return the plain text

StreamChatAsync — the streaming path. Same idea but instead of reading one JSON blob, it reads the HTTP response *line by line* as it arrives:

```
using var stream = await response.Content.ReadAsStreamAsync(ct);
using var reader = new StreamReader(stream);

while (true)
{
    var line = await reader.ReadLineAsync(ct);
    if (line is null) break;
    if (!line.StartsWith("data: ")) continue; // SSE format – only lines starting "data: "
    // ...parse the JSON after "data: ", figure out which of 4 event types it is, yield a ChatC
}
```

Claude's streaming protocol (called Server-Sent Events, or SSE) sends four kinds of events in order, and the code has a `switch` on each: `message_start` (carries the input token count), `content_block_delta` (one new word/piece of text), `message_delta` (carries the stop reason and output token count), `message_stop` (the end — `yield break`).

`TryExtractText`, `TryExtractErrorMessage`, `TryExtractErrorCode`, `TryExtractUsage` — four small private helper methods, all following the same "open the JSON like a tree, ask politely if a branch exists, never throw if it doesn't" pattern described in §3's `JsonDocument` section. `TryExtractUsage` in

particular has to handle *two different shapes* Anthropic has used for cache token counts over time — a flat field and a newer nested `cache_creation` object — and checks both.

Services/Claude/ClaudeBatchApiClient.cs

The batch world's version of the phone — three methods matching the three batch endpoints (`SubmitAsync` , `GetStatusAsync` , `GetResultsAsync`). The submit method builds one request per prompt with a `custom_id` like `"request-0"` , `"request-1"` so results can be matched back up later. `GetResultsAsync` reads the response as **JSONL** — one independent JSON object per line, not one big array — because that's the format batch results come back in, and parses each line into a `BatchResult` . It also estimates the dollar savings from using batch (half price) and logs that number.

Services/Claude/ClaudeProviderException.cs

A custom-built error envelope. A normal .NET exception just carries a message. This one is shaped specifically for "something went wrong talking to Claude" and carries four extra labeled pieces of information so whoever catches it (the global handler in `Program.cs`) can decide exactly what to tell the caller.

```
public sealed class ClaudeProviderException : Exception
{
    public string Provider { get; }           // always "anthropic" today
    public HttpStatusCode? ProviderStatusCode { get; } // what Claude actually returned, if any
    public string? ProviderErrorCode { get; } // Claude's own error type string
    public bool IsTransient { get; }         // "worth retrying later?"
}
```

4.7 Middleware, Extensions, Telemetry — the helpers

Middleware/CorrelationIdMiddleware.cs

Imagine every request gets handed a numbered ticket the moment it walks in the door, like at a deli counter. If the caller already brought a ticket number (an `x-correlation-id` header), keep it. If not, print a new one (`Guid.NewGuid().ToString("N")`). Stamp that number onto the response too, and onto every single log line written while handling this request — so later, if something goes wrong, you can search the logs for one ticket number and see the entire story of that one request.

Extensions/HttpContextExtensions.cs

A tiny convenience. Without it, getting the ticket number back out would need a clunky lookup every time. This file bolts a brand-new method, `.GetCorrelationId()`, directly onto ASP.NET's built-in `HttpContext` type — even though Microsoft wrote that type, not this project. That's what `static class + this HttpContext context` as the first parameter does in C#: it's called an **extension method**.

```
public static string? GetCorrelationId(this HttpContext context) // "this" makes it callable a
```

Telemetry/GatewayTelemetry.cs

One static class holding every metric the whole app records, defined exactly once so nobody accidentally creates the same counter twice under a slightly different name. Two kinds of instrument: a `Counter<long>` only ever goes up by whole numbers (good for "how many requests"), a `Histogram<double>` records a spread of measurements you'll want percentiles of later (good for "how many milliseconds did this take").

5. Three complete journeys, step by step

Journey A — a normal chat question (`POST /api/ai/chat`)

1. Browser/Postman sends `{"prompt": "What's the capital of France?"}`
2. `CorrelationIdMiddleware` stamps a tracking number on the request
3. `AiController.Chat(...)` checks the prompt isn't empty/too long
4. Calls `_provider.SendAsync(request, ct)` — `_provider` is actually a `ClaudeChatModelProvider`
5. `ClaudeChatModelProvider.SendAsync` builds the Claude-shaped JSON, calls `_claudeApiClient.SendChatAsync(payload, ct)`
6. `ClaudeApiClient.SendChatAsync` adds the system prompt + cache settings, `POST` s to `api.anthropic.com/v1/messages`
7. Claude replies with a JSON blob; `TryExtractText` pulls the answer out, `TryExtractUsage` pulls the token counts out
8. Back up through every layer: API client → provider → controller → `Ok(response)` → browser sees `{"provider": "anthropic", "model": "claude-sonnet-4-6", "response": "Paris."}`

Journey B — a streaming question (`POST /api/ai/chat/stream`)

- 1–4. Same as Journey A, but the controller method is `StreamChat` and it disables response buffering and sets `Content-Type: text/event-stream` before writing anything
5. `_provider.StreamAsync(...)` calls `_claudeApiClient.StreamChatAsync(...)`, which opens the connection with `HttpCompletionOption.ResponseHeadersRead` (don't wait for the body, just the headers, so reading can start immediately)
6. Claude sends back text one Server-Sent-Event line at a time; `ClaudeApiClient` turns each useful line into a `ChatChunk` and `yield return` s it instantly
7. `ClaudeChatModelProvider.StreamAsync` records the time-to-first-token the moment chunk #1 arrives, then just passes every chunk through
8. `AiController.StreamChat` writes each chunk to the browser as `data: {...}\n\n` and flushes immediately — the browser sees words appear as they're generated, not all at once at the end

Journey C — a batch job (submit → poll → retrieve)

1. `POST /api/ai/batch` with a list of prompts → `AiBatchController.Submit` → `ClaudeBatchChatModelProvider.SubmitBatchAsync` → `ClaudeBatchApiClient.SubmitAsync` → Claude hands back a batch ID immediately (work hasn't started yet)
2. Sometime later: `GET /api/ai/batch/{id}` → `GetStatus` → `GetBatchStatusAsync` → `GetStatusAsync` → Claude reports how many succeeded/errored/canceled/expired so far
3. Once finished: `GET /api/ai/batch/{id}/results` → `GetResults` → `GetBatchResultsAsync` → `GetResultsAsync` reads the JSONL line by line and returns every answer, matched back to its original prompt by `custom_id`

Notice there's no retry anywhere in this path — on purpose. If the network blips while submitting, the submit might have actually gone through on Claude's side already; retrying could create and pay for a duplicate batch. ADR-010 explains this reasoning in full.

6. How pieces find each other (Dependency Injection)

Nobody in this codebase ever writes `new ClaudeChatModelProvider(...)` by hand (except in test code, deliberately). Instead, `Program.cs` registers a master phonebook once at startup — "if anyone asks for an `IChatModelProvider`, hand them a freshly built `ClaudeChatModelProvider`" — and ASP.NET's built-in container looks up that phonebook every time a controller is created, automatically building the whole chain of constructor arguments underneath it.

```
builder.Services.AddScoped<IChatModelProvider, ClaudeChatModelProvider>();
```

When code asks for...	The container hands back...	Which itself needed...
<code>IChatModelProvider</code>	<code>ClaudeChatModelProvider</code>	<code>ClaudeApiClient</code> , <code>IOptions<AnthropicOptions></code> , <code>ILogger<ClaudeChatModelProvider></code>
<code>ClaudeApiClient</code>	itself (registered via <code>AddHttpClient<ClaudeApiClient></code>)	a pre-configured <code>HttpClient</code> with the API key already in its headers
<code>IBatchChatModelProvider</code>	<code>ClaudeBatchChatModelProvider</code>	<code>ClaudeBatchApiClient</code>
<code>IOptions<AnthropicOptions></code>	the bound <code>"Anthropic"</code> section of <code>appsettings.json</code>	(read once at startup)

`AiController` 's constructor asks for `IChatModelProvider`, `ILogger<AiController>`, and `IOptions<AnthropicOptions>` — the container builds all three (and everything *they* need) without the controller ever knowing a concrete class name like `ClaudeChatModelProvider` exists. This is what makes swapping providers later a one-line change in `Program.cs`, nowhere else.

7. Glossary — every C# word used in this project

namespace

A folder-like label that groups related code and prevents naming collisions.

using

"I'm borrowing code from that namespace" — lets you write short names instead of full paths.

class

A blueprint for an object that can hold data and do things.

interface

A list of promises a class can agree to keep, with no implementation of its own.

record

A class built for holding immutable data, with free equality comparison.

enum

A fixed list of named choices (e.g. `InProgress` , `Ended`).

sealed

"No one is allowed to build a further specialized version of this class."

static

Belongs to the type itself, not to any one instance of it — there's only ever one copy.

readonly

Can be set once (usually in the constructor) and never changed again.

public / private

Who's allowed to see/use this: everyone, or only code inside the same class.

property ({ get; set; })

A labeled, readable/writable slot on an object.

constructor

The special method that runs the moment an object is created, usually to receive its tools.

async / await

Marks a method that might have to wait, and the point where it actually pauses to wait.

Task<T>

A promise of a value of type T, to be delivered later.

IAsyncEnumerable<T>

A promise of a whole stream of T values, arriving one at a time over time.

yield return

"Here's the next item in the stream — pause here until someone asks for the next one."

CancellationToken

A signal passed down through every layer meaning "stop what you're doing if I tell you to" — used when a caller disconnects mid-stream.

nullable (?)

Admits this value might be missing/empty; the compiler forces you to check.

null-conditional (?.)

"Do this, but only if the thing on the left isn't missing."

null-coalescing (??)

"Use this value, or this fallback if it's missing."

string interpolation (\$"...")

A sentence with blanks filled in by code, like `"got {count} results"`.

switch expression

A sideways lookup table: match a value, get back a result, with `_` as the catch-all.

try / catch / throw / finally

Attempt something; handle specific failures; raise a new failure; always run cleanup code regardless of success or failure.

exception

An object representing "something went wrong," thrown up through the call stack until something catches it.

attribute ([ApiController])

A sticky note on code that tells the framework to treat it specially.

generics (<T>)

A container shape that works for any type, decided when you use it.

LINQ (.Select(...) , .Any(...))

Built-in shorthand for transforming or checking a whole collection in one line instead of writing a loop.

Dependency Injection (DI)

Letting a container build and hand objects their required tools automatically, instead of constructing everything by hand.

middleware

Code that runs on every request, in a chain, before/after the actual controller — like checkpoints in a hallway.

extension method

A method that appears to be bolted onto a type you don't own, via a special `this` first parameter.

JsonDocument

A way to read JSON as a flexible tree and ask "does this branch exist?" instead of forcing a fixed shape.

ActivitySource / Histogram / Counter

OpenTelemetry building blocks for tracing (a labeled timeline of work) and metrics (counted/measured numbers over time).